

OBJECT ORIENTED ANALYSIS DESIGN LAB MANUAL

TASK 1:

INTRODUCTION TO UML (UNIFIED MODELING LANGUAGE):

The UML is a language for specifying, constructing, visualizing, and documenting the software system and its components. The UML is a graphical language with sets of rules and semantics. The rules and semantics of a model are expressed in English in a form known as OCL (Object Constraint Language). OCL uses simple logic for specifying the properties of a system. The UML is not intended to be a visual programming language. However, it has a much closer mapping to object-oriented programming languages, so that the best of both can be obtained. The UML is much simpler than other methods preceding it. UML is appropriate for modeling systems, ranging from enterprise information system to distributed web-based application and even to real time embedded system. It is a very expensive language addressing all views needed to develop and then to display system even though understand to use. Learning to apply UML effectively starts forming a conceptual mode of languages which requires learning.

Three major language elements:

- UML basic building blocks.
- Rules that dictate how this building blocks put together.
- Some common mechanism that apply throughout the language.

Every model can be expressed at different levels of fidelity. The best models are connected to reality.

The UML defines nine graphical diagrams:

1. Class diagram
2. Use-case diagram
3. Behavior diagram
 - 3.1. Interaction diagram
 - 3.1.1. sequence diagram
 - 3.1.2. collaboration diagram
 - 3.2. state chart diagram
 - 3.3. activity diagram
4. Implementation diagram
 - 4.1 component diagram
 - 4.2 deployment diagram

1. UML class diagram: The UML class diagram is also known as object modeling. It is a static analysis diagram. These diagrams show the static structure of the model. A class diagram is a connection of static model elements, such as classes and their relationships, connected as a graph to each other and to their contents.

2. Use-case diagram: The functionality of a system can be described in a number of different use-cases, each of which represents a specific flow of events in a system. It is a graph of actors, a set of use-cases enclosed in a boundary, communication, associations between the actors and the use-cases, and generalization among the use-cases.

3. Behavior diagram: It is a dynamic model unlike all the others mentioned before. The objects of an object-oriented system are not static and are not easily understood by static diagrams. The behavior of the class's instance (an object) is represented in this diagram. Every use-case of the system has an associated behavior diagram that indicates the behavior of the object. In conjunction with the use-case diagram we may provide a script or interaction diagram to show a time line of events. It consists of sequence and collaboration diagrams.

4. Interaction diagram: It is the combination of sequence and collaboration diagram. It is used to depict the flow of events in the system over a timeline. The interaction diagram is a dynamic model which shows how the system behaves during dynamic execution.

5. State chart diagram: It consists of state, events and activities. State diagrams are a familiar technique to describe the behavior of a system. They describe all of the possible states that a particular object can get into and how the object's state changes as a result of events that reach the object. In most OO techniques, state diagrams are drawn for a single class to show the lifetime behavior of a single object.

6. Activity diagram: It shows organization and their dependence among the set of components. These diagrams are particularly useful in connection with workflow and in describing behavior that has a lot of parallel processing. An activity is a state of doing something: either a real-world process, or the execution of a software routine.

7. Implementation diagram: It shows the implementation phase of the systems development, such as the source code structure and the run-time implementation structure. These are relatively simple high-level diagrams compared to the others seen so far. They are of two sub-diagrams, the component diagram and the deployment diagram.

8. Component diagram: These are organizational parts of a UML model. These are boxes to which a model can be decomposed. They show the structure of the code itself. They model the physical components such as source code, user interface in a design. It is similar to the concept of packages.

9. Deployment diagram: The deployment diagram shows the structure of the runtime system. It shows the configuration of runtime processing elements and the software

components that live in them. They are usually used in conjunction with deployment diagrams to show how physical modules of code are distributed on the system.

NOTATION ELEMENTS: These are explanatory parts of UML model. They are boxes which may apply to describe and remark about any element in the model. They provide the information for understanding the necessary details of the diagrams.

Relations in the UML: These are four kinds of relationships used in an UML diagram, they are:

1. Dependency
 2. Association
 3. Generalization
 4. Realization
1. **Dependency:** It is a semantic relationship between two things in which a change one thing affects the semantics of other things. Graphically a dependency is represented by a non-continuous line.
 2. **Association:** It is a structural relationship that describes asset of links. A link is being connected among objects. Graphically association is represented as a solid line possibly including label.
 3. **Generalization:** It is a specialized relationship in which the specialized elements are substitutable for object of the generalized element. Graphically it is a solid line with hollow arrow head parent.
 4. **Realization:** It is a semantic relation between classifiers. Graphically it is represented as a cross between generalization and dependency relationship.

TASK 2: PASSPORT AUTOMATION SYSTEM

AIM: To create an automated system to perform the Passport Process.

PROBLEM STATEMENT:

Passport Automation System is used in the effective dispatch of passport to all of the applicants. This system adopts a comprehensive approach to minimize the manual work and schedule resources, time in a cogent manner. The core of the system is to get the online registration form (with details such as name, address etc.,) filled by the applicant whose testament is verified for its genuineness by the Passport Automation System with respect to the already existing information

SOFTWARE REQUIREMENT SPECIFICATION:

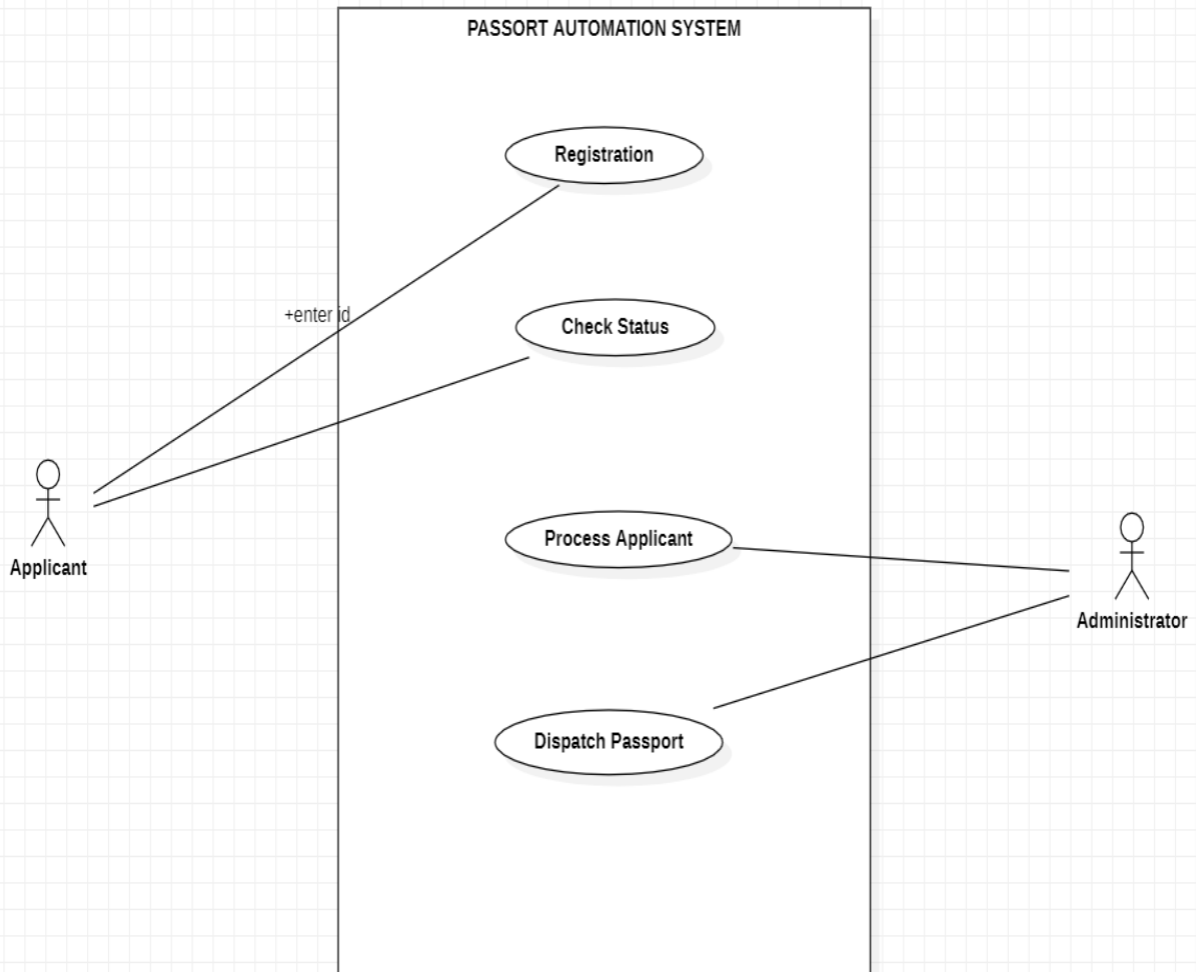
SOFTWARE INTERFACE

- Front End Client - The applicant and Administrator online interface is built using JSP and HTML. The Administrator's local interface is built using Java.
- Web Server - Glassfish application server (Oracle Corporation).
- Back End - Oracle database.

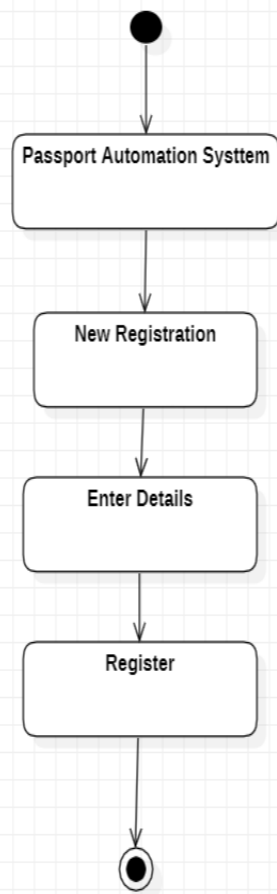
HARDWARE INTERFACE

The server is directly connected to the client systems. The client systems have access to the database in the server.

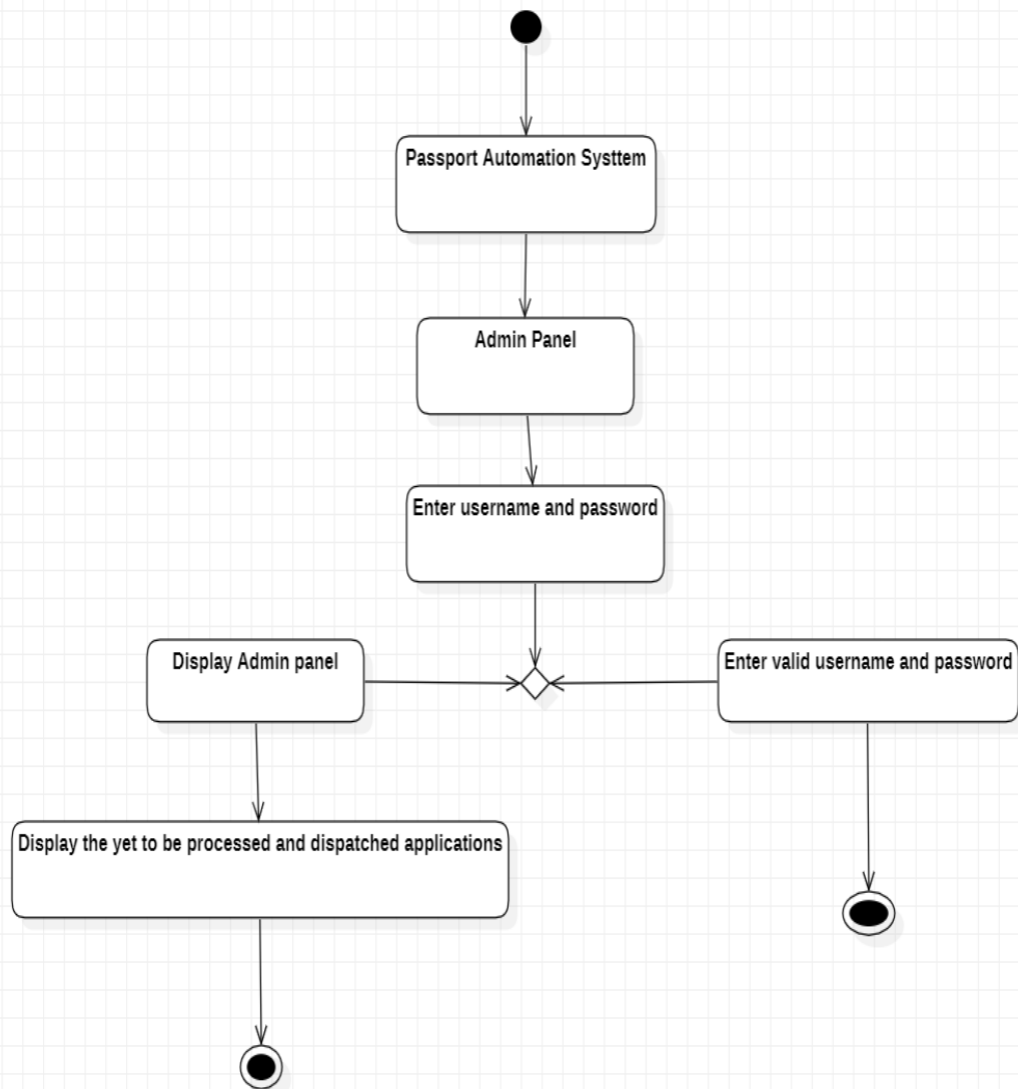
USECASE DIAGRAM FOR PASSPORT AUTOMATION SYSTEM



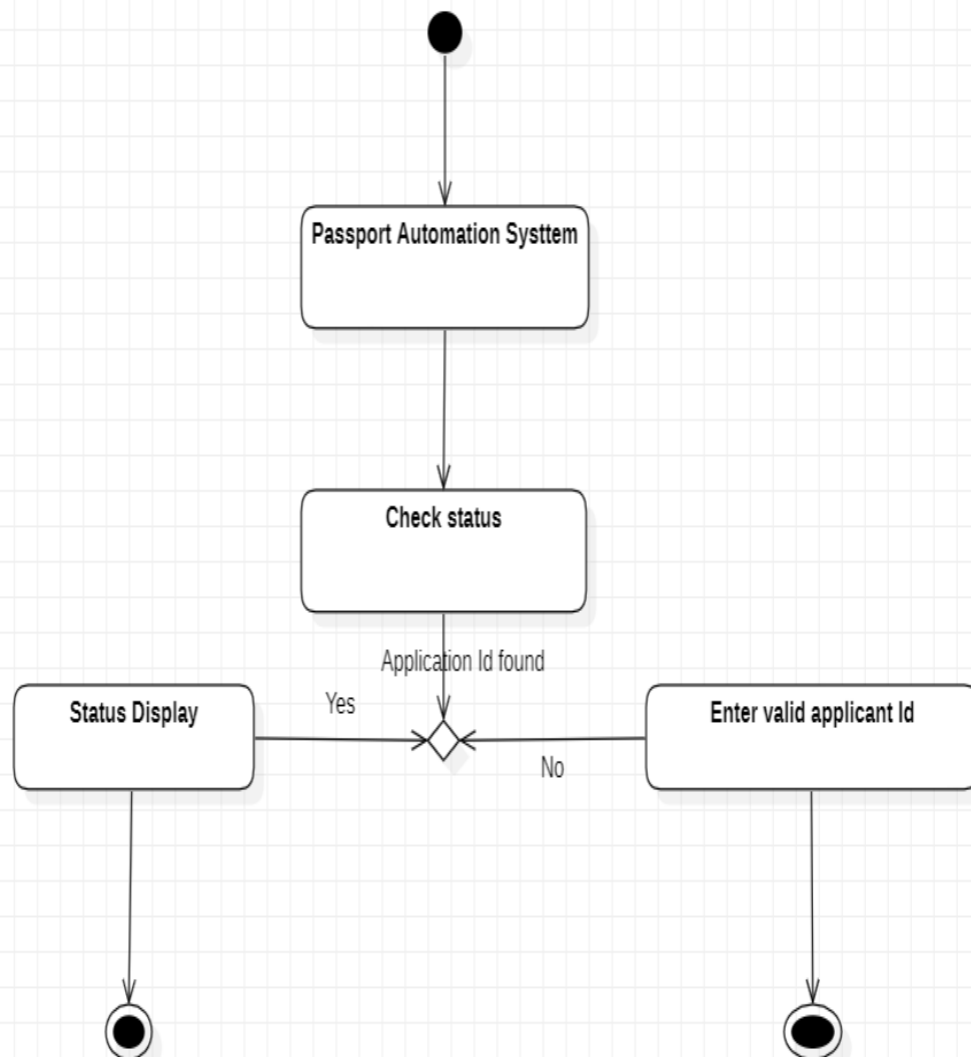
ACTIVITY DIAGRAM FOR REGISTER



ACTIVITY DIAGRAM FOR ADMINISTRATION



ACTIVITY DIAGRAM FOR CHECKING STATUS



CLASS DIAGRAM

The class diagram, also referred to as object modeling is the main static analysis diagram. The main task of object modeling is to graphically show what each object will do in the problem domain. The problem domain describes the structure and the relationships among objects.

The Passport Automation system class diagram consists of four classes. Passport Automation System

1.New registration

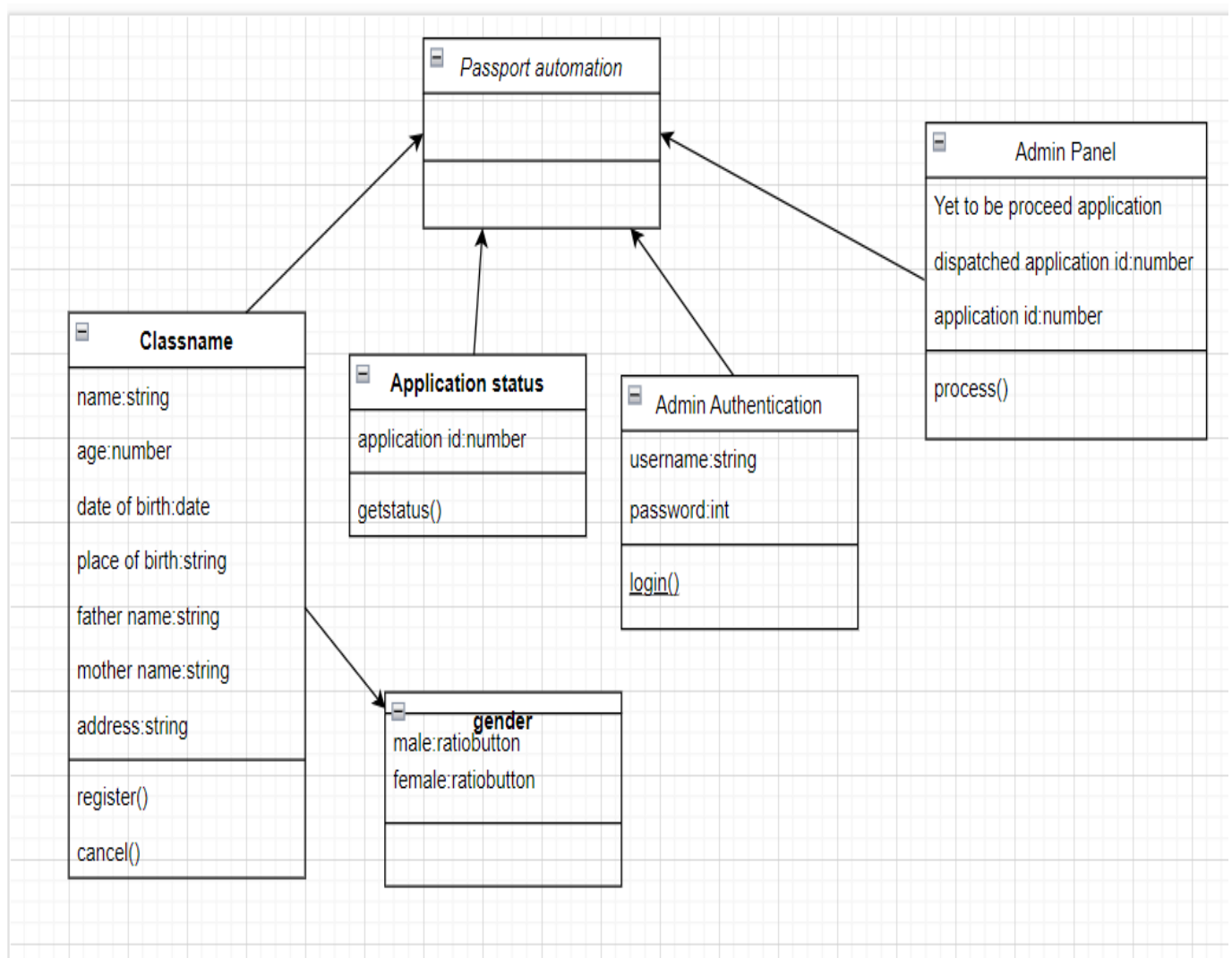
2.Gender

3.Application Status

4. Admin authentication

5. Admin Panel

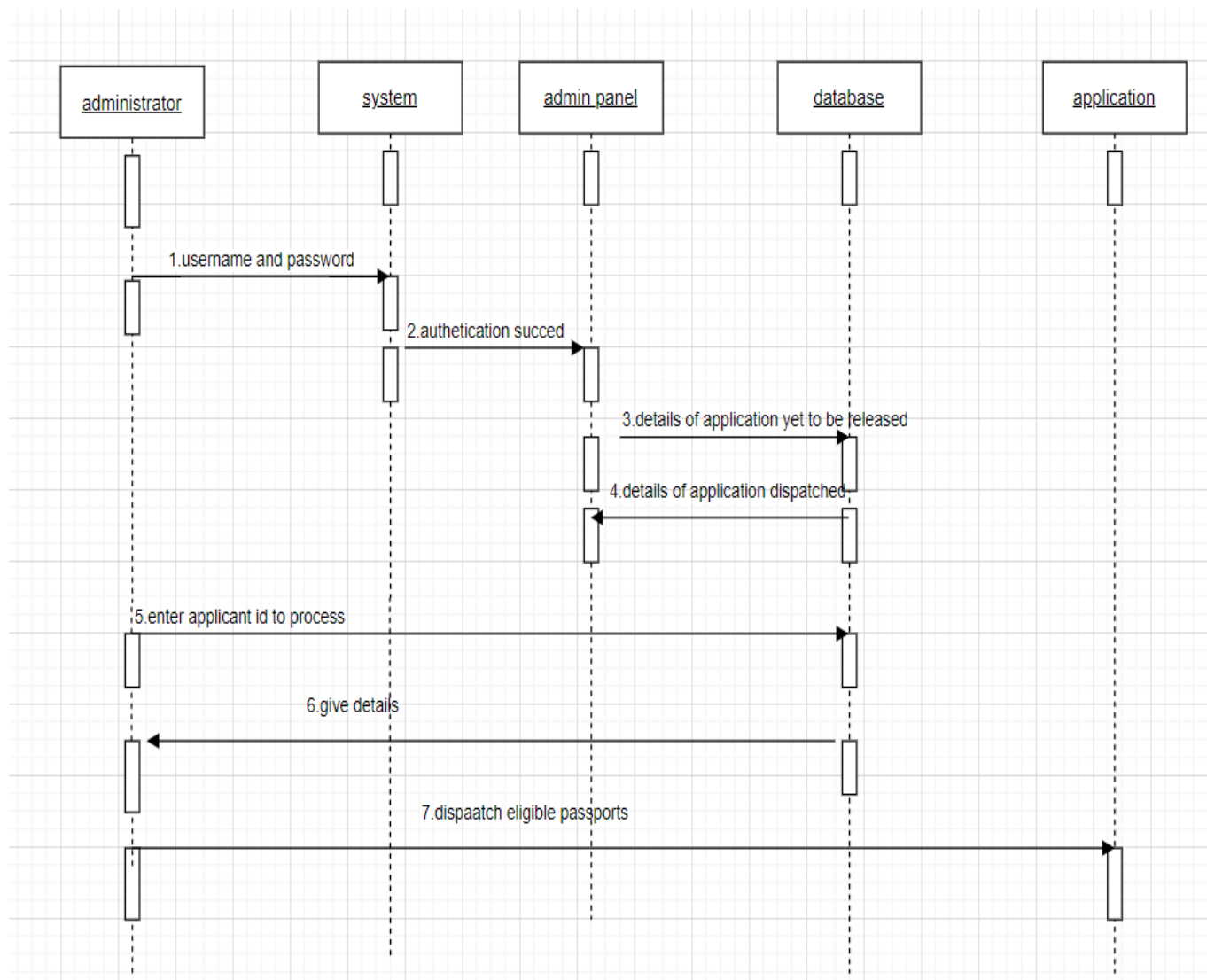
CLASS DIAGRAM FOR PASSPORT AUTOMATION SYSTEM



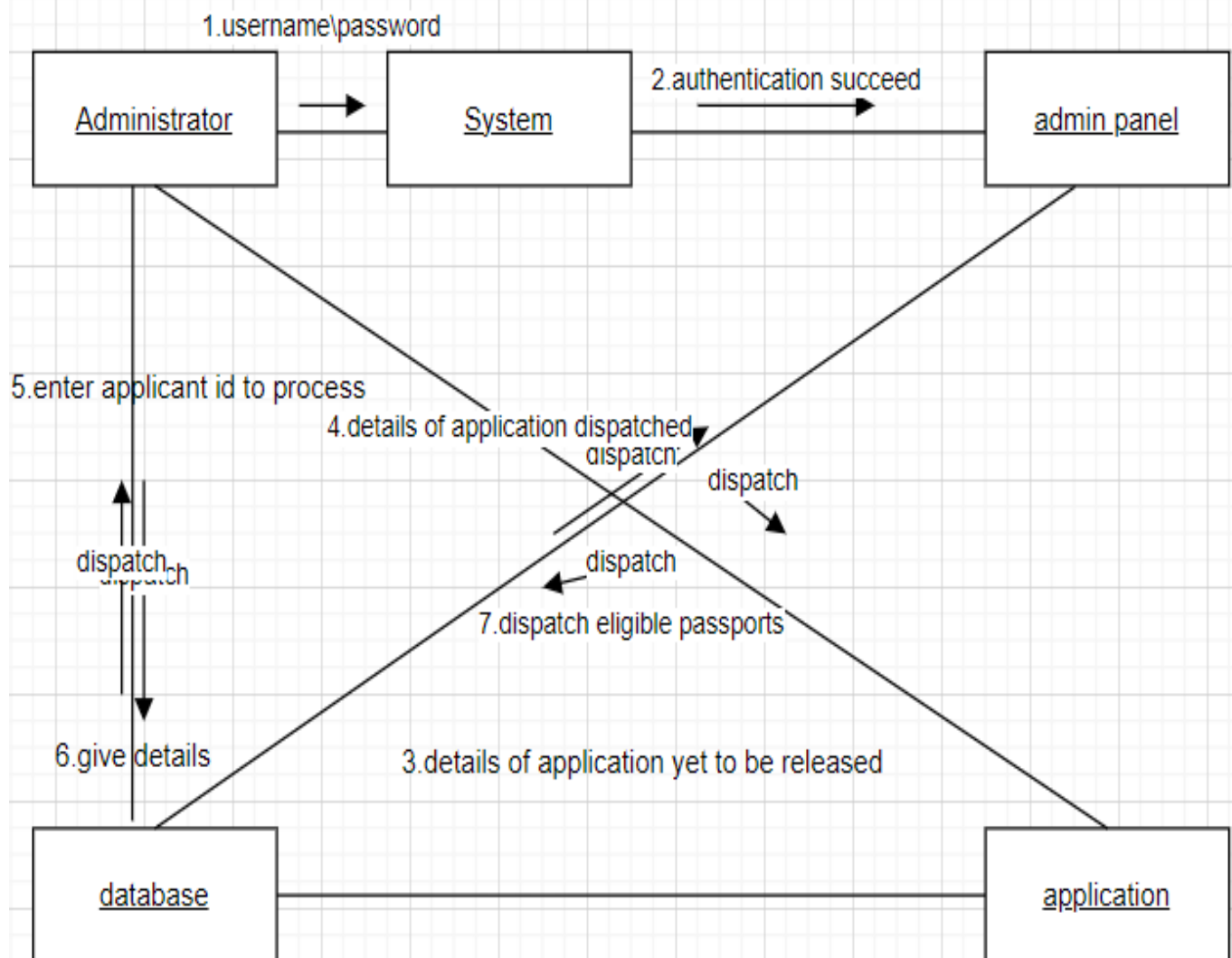
(VI) INTERACTION DIAGRAM: A sequence diagram represents the sequence and interactions of a given USE-CASE or scenario. Sequence diagrams can capture most of the information about the system. Most object-to-object interactions and operations are considered events and events include signals, inputs, decisions, interrupts, transitions and actions to or from users or external devices.

It is also represented by the order in which things occur and how the objects in the system send message to one another. The sequence diagram for each USE-CASE that exists when a user administrator, check status and new registration about passport automation system are given.

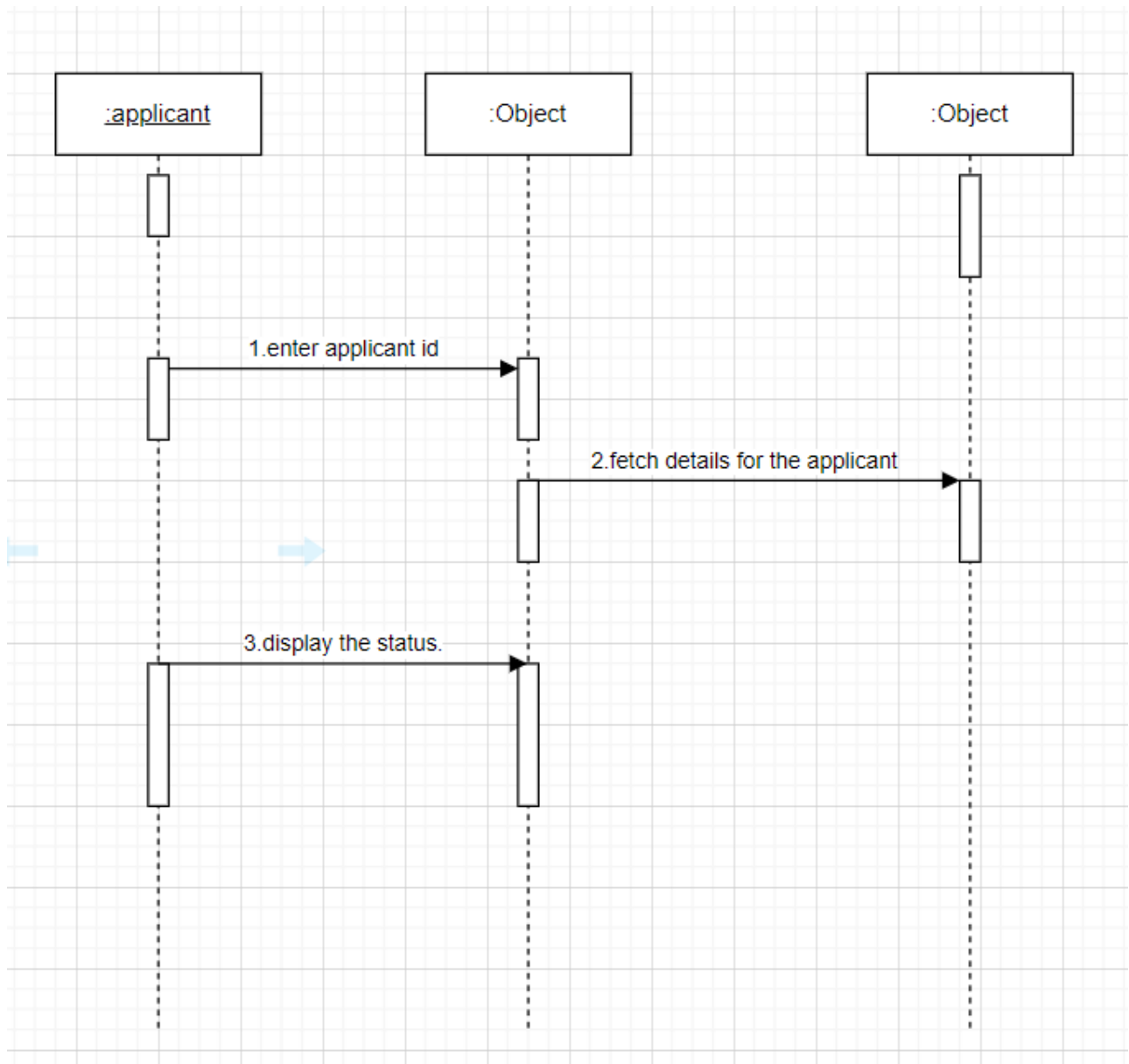
SEQUENCE DIAGRAM FOR ADMINISTRATOR



COLLABORATION DIAGRAM FOR ADMINISTRATOR

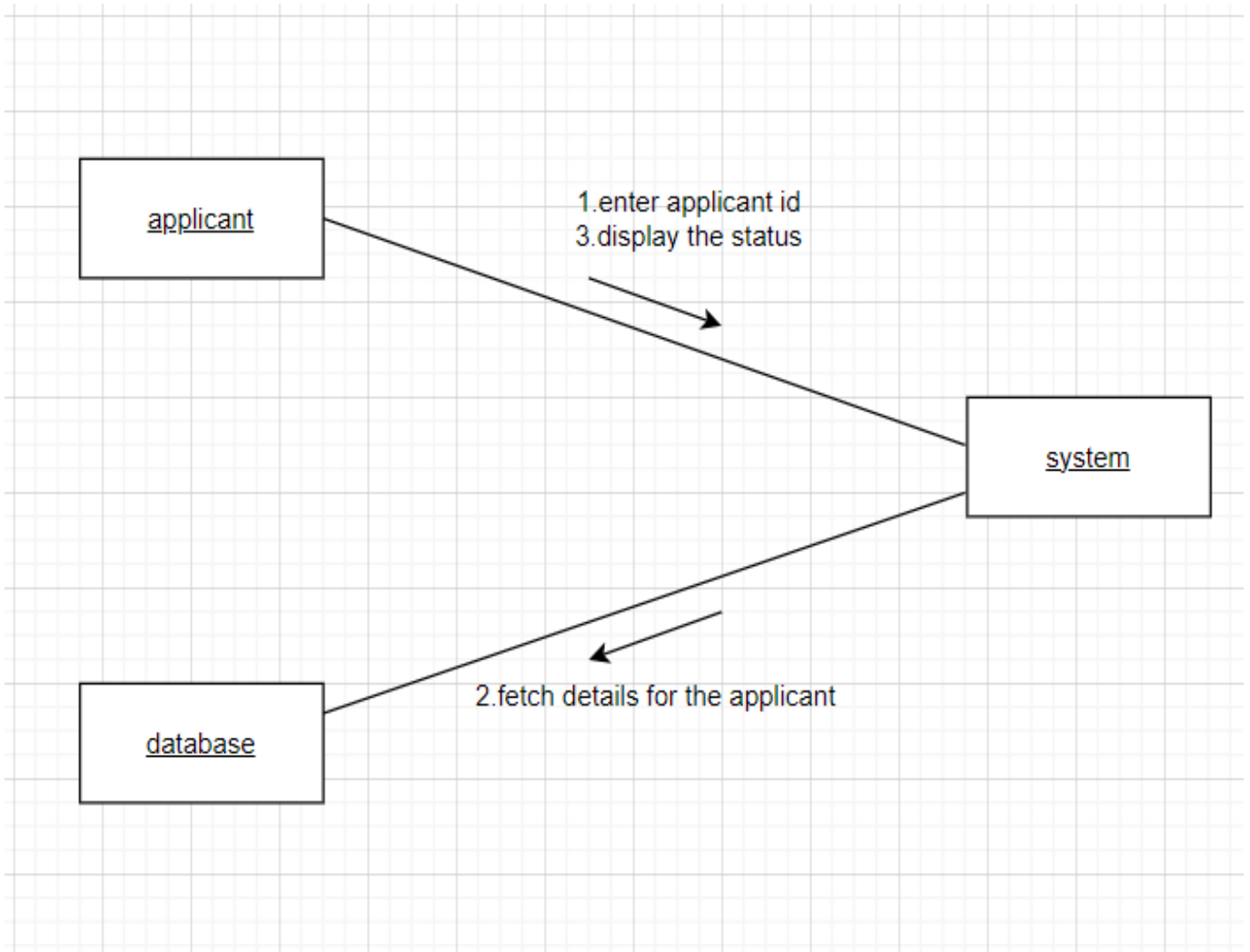


SEQUENCE DIAGRAM FOR CHECKING STATUS

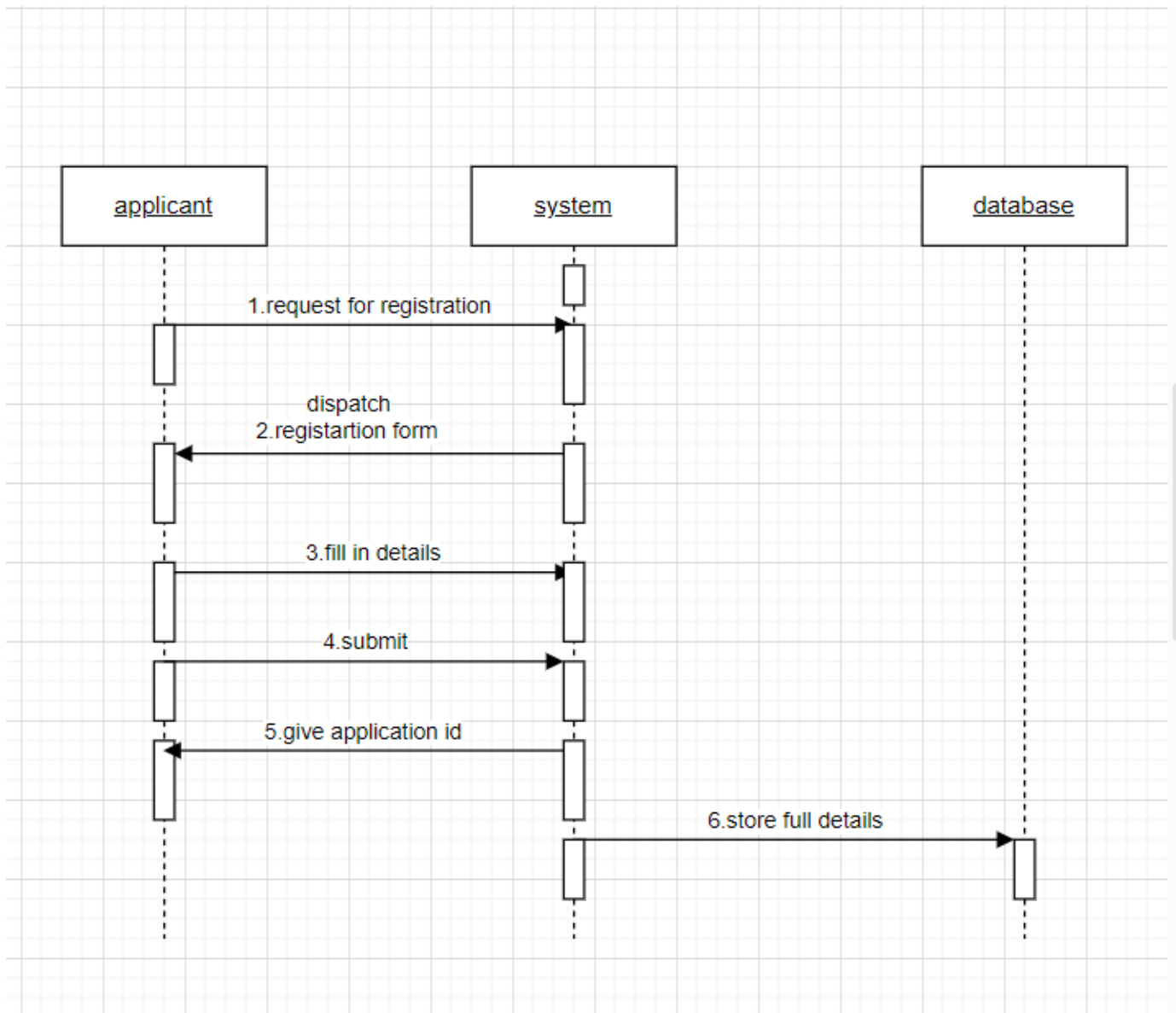


COLLABORATION DIAGRAM FOR CHECKING STATUS

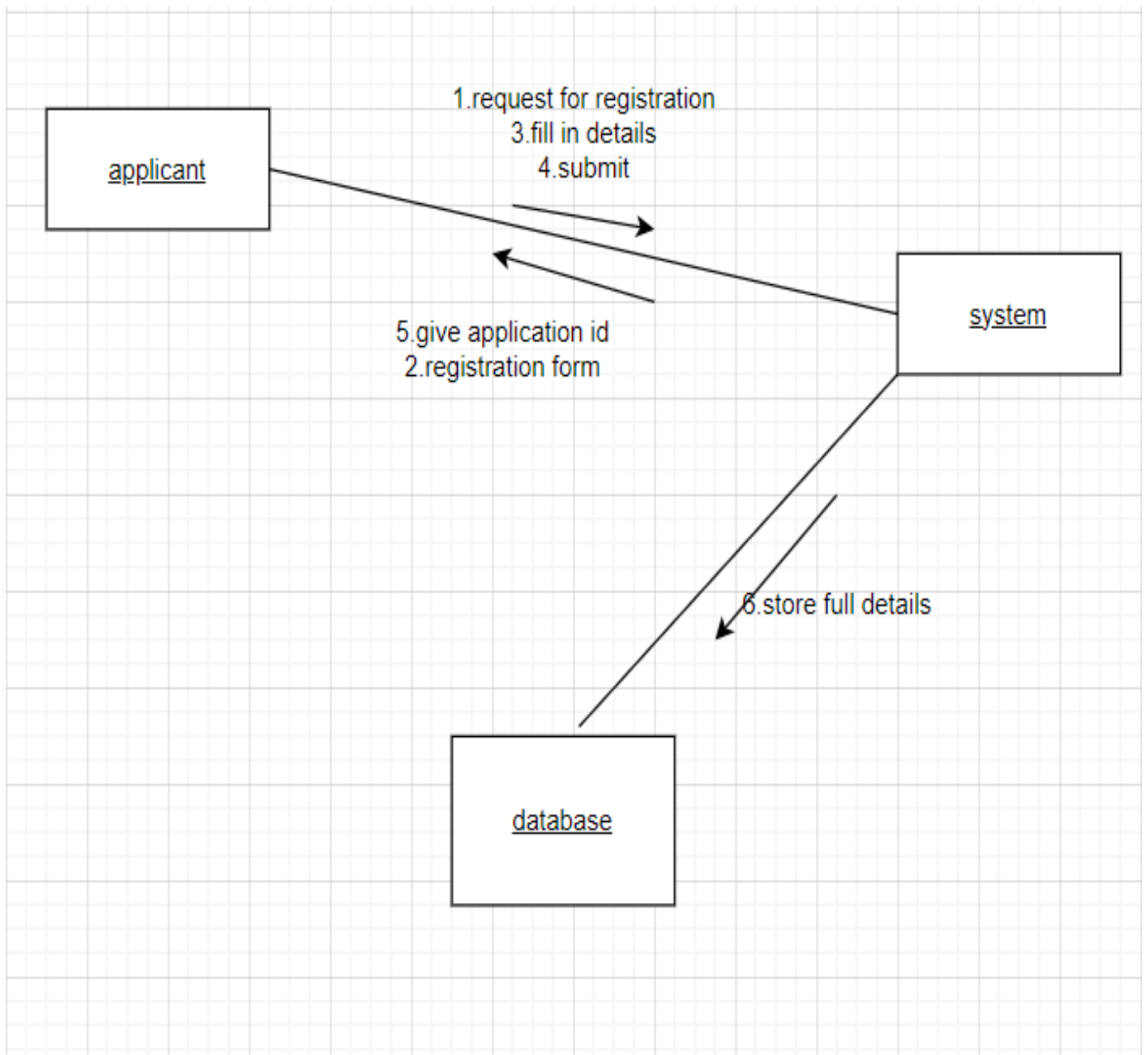
The diagrams show the applicant enters his id and the system fetch the details from the database and display the status.



SEQUENCE DIAGRAM FOR NEW REGISTRATION

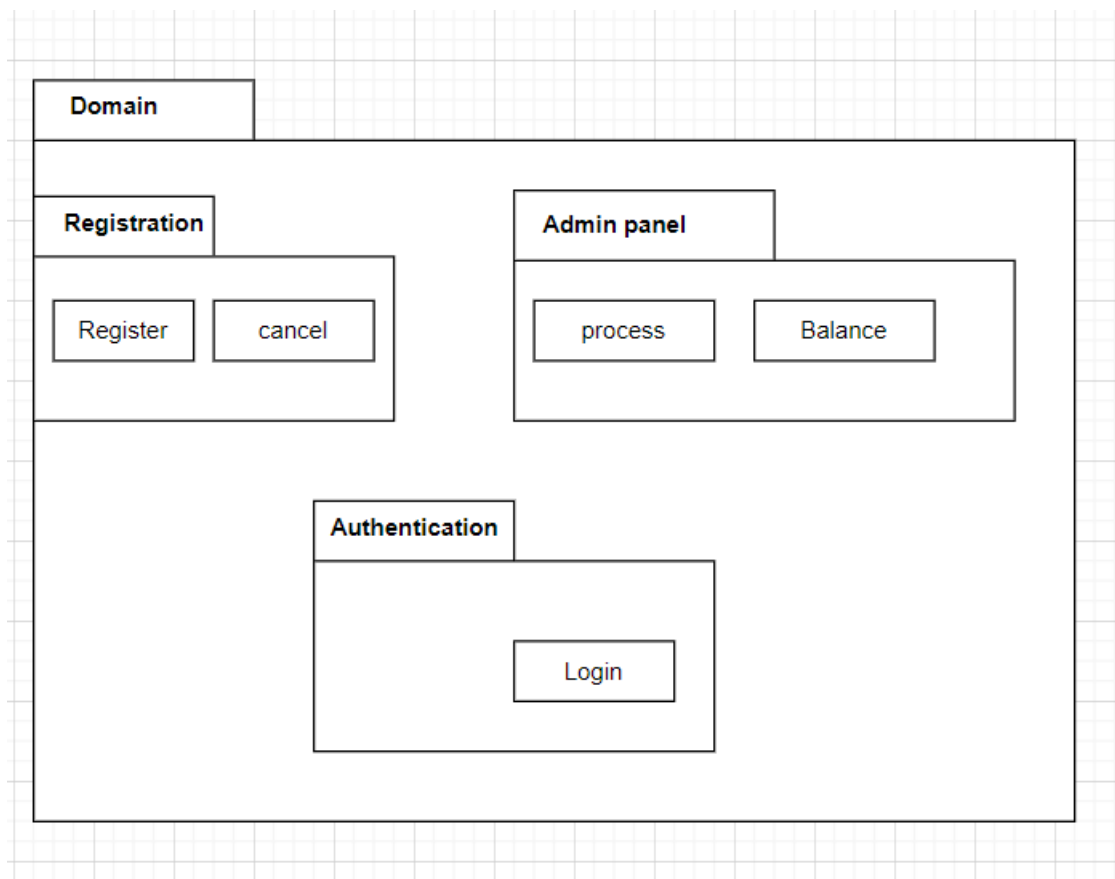
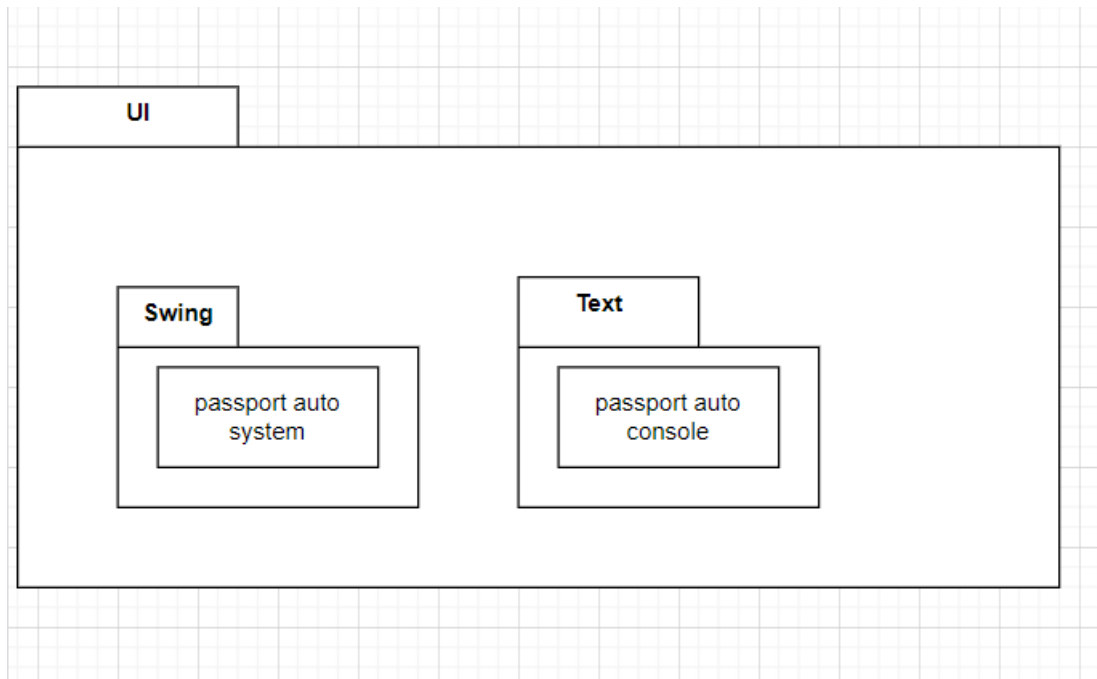


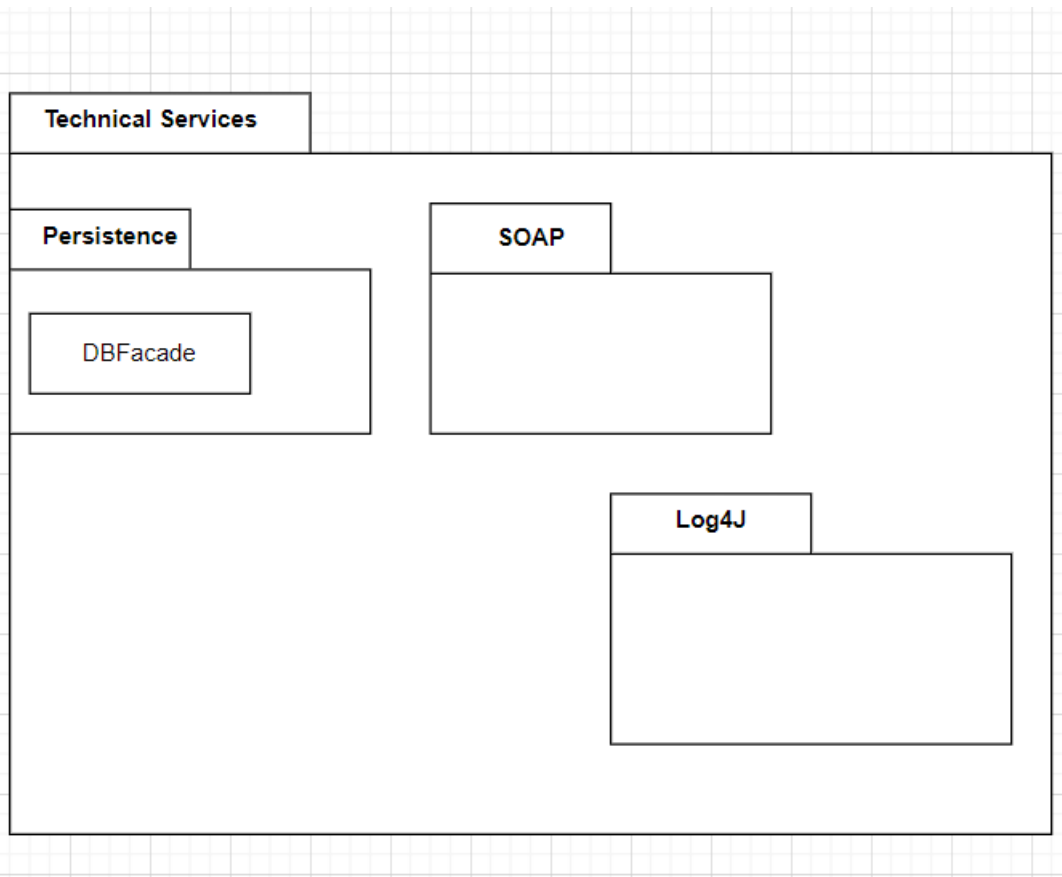
COLLABORATION DIAGRAM FOR NEW REGISTRATION



The diagrams show the applicant request the system for registration and the system provide the register form and applicant fill the form and submit and the system give the applicant id. The database stores the full details.

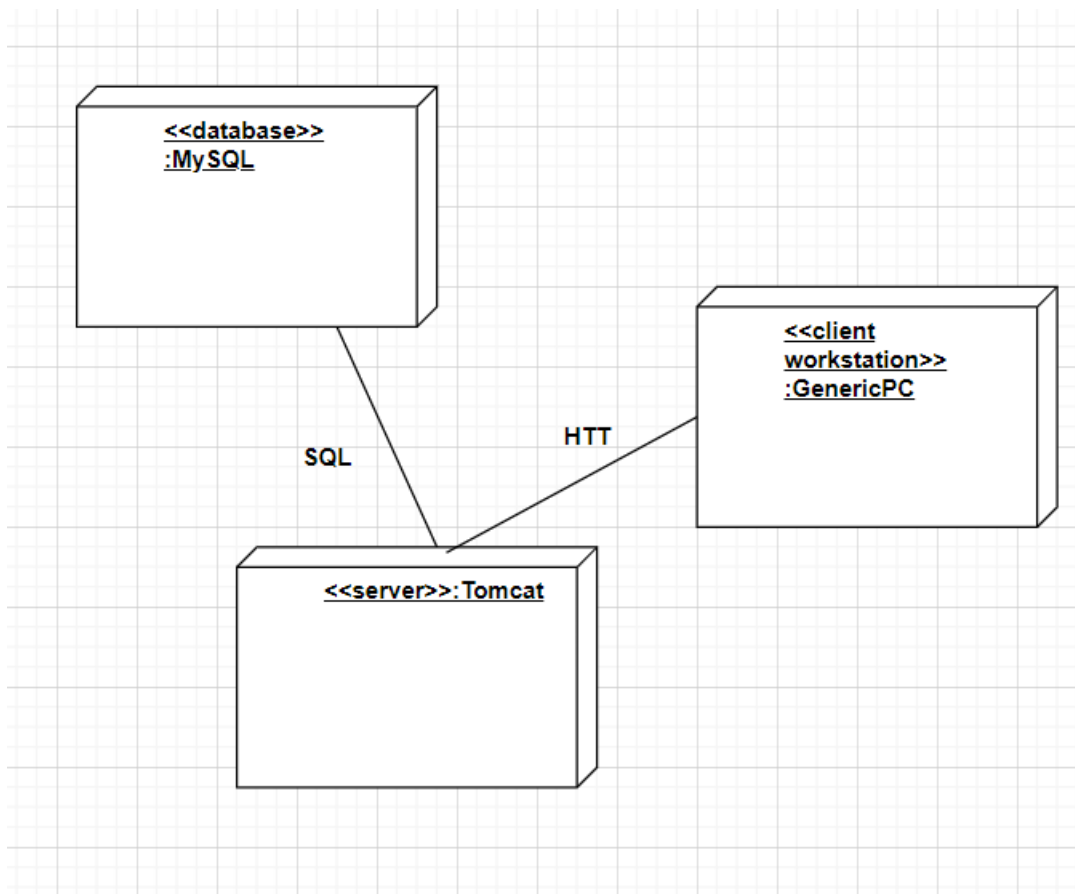
PARTIAL LAYERD LOGICAL ARCHITECTURE DIAGRAM





DEPLOYMENT DIAGRAM AND COMPONENT DIAGRAM

Deployment diagrams are used to visualize the topology of the physical components of a system where the software components are deployed.



COMPONENT DIAGRAM

Component diagrams are used to visualize the organization and relationship among components in system.

